

A palavra de controle (CON) — na prática

Guia de estudo do sinal mais importante do controlador do SAP-1: a **palavra de controle** de 12 bits (`con`). Ela é a "receita" de cada estado.

1. O que é: 12 interruptores, um por sinal

A cada estado (T1–T6), o controlador gera **12 bits de uma vez** — cada bit **liga ou desliga** um sinal de controle. É um painel de 12 interruptores que muda a cada estado.

```
CON = { Cp, Ep, ~Lm, ~CE, ~Li, ~Ei, ~La, Ea, Su, Eu, ~Lb, ~Lo }
bit:  11 10  9  8  7  6  5  4  3  2  1  0
```

Cada bit comanda um bloco do processador (ligar a saída no barramento, ou mandar carregar):

Bit	Sinal	O que faz quando ativo
11	Cp	PC incrementa (+1)
10	Ep	PC joga seu valor no barramento
9	~Lm	MAR carrega do barramento
8	~CE	RAM joga o dado no barramento
7	~Li	IR carrega do barramento
6	~Ei	IR joga o operando (endereço) no barramento
5	~La	Acumulador A carrega do barramento
4	Ea	A joga seu valor no barramento
3	Su	ALU subtrai (em vez de somar)
2	Eu	ALU joga o resultado no barramento
1	~Lb	Registrador B carrega do barramento
0	~Lo	Registrador de saída carrega do barramento

2. A pegadinha: nem todo "ativo" é 1

Há dois tipos de sinal:

Tipo	Ativo quando	Quais
Ativo-alto (sem til)	vale 1	Cp, Ep, Ea, Su, Eu
Ativo-baixo (com til ~)	vale 0	~Lm, ~CE, ~Li, ~Ei, ~La, ~Lb, ~Lo

Por isso o estado **parado** (IDLE), onde ninguém faz nada, **não é** `0000...`. É:

```
IDLE = 0011_1110_0011
```

Os ativos-alto ficam **0** (desligados) e os ativos-baixo ficam **1** (desligados). Guarde esse valor: é o "repouso".

3. Decodificando um de verdade: T1

No código: `con = 12'b0101_1110_0011`. Abrindo bit a bit:

```
Cp Ep ~Lm ~CE ~Li ~Ei ~La Ea Su Eu ~Lb ~Lo
valor: 0 1 0 1 1 1 1 0 0 0 1 1
      | | |
      | | └─ ~Lm = 0 -> ATIVO! (MAR carrega)
      | └─── Ep = 1 -> ATIVO! (PC joga no barramento)
      └───── Cp = 0 -> desligado
```

Resultado: `Ep` liga a saída do PC no barramento e `~Lm` manda o MAR carregar → **MAR ← PC**. Só esses dois; o resto em repouso.

4. O truque de leitura: compare com o IDLE

```
IDLE = 0011_1110_0011
T1    = 0101_1110_0011
      ↑↑
      diferem em Ep (0→1) e ~Lm (1→0)
```

Para ler qualquer CON, compare com o IDLE. Os bits que **diferem** são exatamente os sinais ativos naquele estado. É o jeito mais rápido de entender uma palavra de controle.

5. Mais exemplos reais

T3 — `0010_0110_0011` (busca da instrução):

```
~CE = 0 (ATIVO: RAM joga no barramento)
~Li = 0 (ATIVO: IR carrega)
-> IR ← RAM
```

T6 do ADD — `0011_1100_0111` (a soma):

```
~La = 0 (ATIVO: A carrega)
Eu = 1 (ATIVO: ALU joga no barramento)
-> A ← (A + B)
```

T6 do SUB — `0011_1100_1111`: igual ao ADD, mas com `Su = 1` também → a ALU subtrai → **A ← (A - B)**.

6. Como isso aparece na onda

Na janela Wave, o sinal `con` (deixado em **binário**) mostra esses 12 bits mudando a cada estado. Leia assim:

1. Olhe o `tstate` (ex.: T1).
2. Olhe o `con` naquele instante (ex.: `0101_1110_0011`).
3. Compare com o IDLE → os bits diferentes são os sinais ativos.
4. Confira: os sinais individuais (`ep`, `lm_bar` ...) logo abaixo do `con` batem com a sua leitura.

7. Tabela completa (todas as receitas)

Estado	CON (binário)	Sinais ativos	Ação
IDLE	<code>0011_1110_0011</code>	—	nada
T1	<code>0101_1110_0011</code>	<code>Ep</code> , <code>~Lm</code>	<code>MAR ← PC</code>
T2	<code>1011_1110_0011</code>	<code>Cp</code>	<code>PC ← PC+1</code>
T3	<code>0010_0110_0011</code>	<code>~CE</code> , <code>~Li</code>	<code>IR ← RAM</code>
T4 (LDA/ADD/SUB)	<code>0001_1010_0011</code>	<code>~Lm</code> , <code>~Ei</code>	<code>MAR ← operando</code>
T4 (OUT)	<code>0011_1111_0010</code>	<code>Ea</code> , <code>~Lo</code>	<code>Saída ← A</code>
T5 (LDA)	<code>0010_1100_0011</code>	<code>~CE</code> , <code>~La</code>	<code>A ← RAM</code>
T5 (ADD/SUB)	<code>0010_1110_0001</code>	<code>~CE</code> , <code>~Lb</code>	<code>B ← RAM</code>
T6 (ADD)	<code>0011_1100_0111</code>	<code>~La</code> , <code>Eu</code>	<code>A ← A+B</code>
T6 (SUB)	<code>0011_1100_1111</code>	<code>~La</code> , <code>Eu</code> , <code>Su</code>	<code>A ← A-B</code>

A ideia central

A palavra de controle é a **receita de cada estado**: 12 bits que dizem exatamente *quem fala e quem ouve* no barramento naquele momento. O programa não muda essa receita — ela depende só do **estado (T1-T6)** e do **opcode**. É isso que faz o processador "funcionar sozinho".

Quem gera essa palavra a cada estado é o **controlador/sequenciador** (`controller_sequencer.v`), explicado em `FSM_explicacao.md`.